

Πολυτεχνική Σχολή

Τμήμα Μηχανικών Η/Υ & Πληροφορικής

Εισαγωγή στη Βιοπληροφορική

Εργασία

7 Σεπτεμβρίου 2026

Περιεχόμενα

1 Ερώτημα 1	3
1.1 Εργαλεία πρόβλεψης πρωτεϊνικών δομών	3
1.2 Αναζήτηση BLASTP και ανάκτηση αλληλουχιών	3
1.3 Πολλαπλές στοιχίσεις και σύγκριση	3
2 Ερώτημα 2	4
2.1 Ταυτοποίηση της πρωτεΐνης στη βάση UniProt	4
2.2 Υλοποίηση	4
2.3 Αποτελέσματα	4
2.4 Απαντήσεις στις ερωτήσεις	4
3 Ερώτημα 3	5
3.1 Υλοποίηση και εκτέλεση	5
3.2 Αποτελέσματα της αναζήτησης	5
3.3 Απαντήσεις στις ερωτήσεις	5
4 Ερώτημα 4	6
4.1 Σύγκριση αλληλουχιών με EMBOSS Needle	6
4.2 Πρόβλεψη και σύγκριση δομών	6
4.3 Προαιρετικά υποερωτήματα	6
5 Ερώτημα 5	7
5.1 Ολική στοίχιση	7
5.1.1 Αναδρομική σχέση	7
5.1.2 Παραδείγματα υπολογισμού	7
5.1.3 Ο πίνακας	8
5.1.4 Οπισθοδρόμηση και στοίχιση	8
5.2 Τοπική στοίχιση	9
5.2.1 Διαφορές από την ολική στοίχιση	9
5.2.2 Ο πίνακας	10
5.2.3 Οπισθοδρόμηση και στοίχιση	10
5.3 Συμπεράσματα	11
6 Ερώτημα 6	12
6.1 Αλγόριθμος με suffix tree	12
6.1.1 Διατύπωση του προβλήματος	12
6.1.2 Βασική ιδιότητα του suffix tree	12
6.1.3 Το εμπόδιο και η άρση του	12
6.1.4 Ο αλγόριθμος	13

6.1.5	Ορθότητα	13
6.1.6	Ανάλυση πολυπλοκότητας	14
6.2	Δεικτοδότηση βιολογικών δεδομένων: Bloom filters, suffix arrays και BWT . .	14
Βιβλιογραφία		15

1 Ερώτημα 1

1.1 Εργαλεία πρόβλεψης πρωτεϊνικών δομών

1.2 Αναζήτηση BLASTP και ανάκτηση αλληλουχιών

1.3 Πολλαπλές στοιχίσεις και σύγκριση

2 Ερώτημα 2

2.1 Ταυτοποίηση της πρωτεΐνης στη βάση UniProt

2.2 Υλοποίηση

2.3 Αποτελέσματα

2.4 Απαντήσεις στις ερωτήσεις

3 Ερώτημα 3

3.1 Υλοποίηση και εκτέλεση

3.2 Αποτελέσματα της αναζήτησης

3.3 Απαντήσεις στις ερωτήσεις

4 Ερώτημα 4

4.1 Σύγκριση αλληλουχιών με EMBOSS Needle

4.2 Πρόβλεψη και σύγκριση δομών

4.3 Προαιρετικά υποερωτήματα

5 Ερώτημα 5

Δίνονται οι ακολουθίες

$$v = \text{ACTTGGGTG}, \quad w = \text{GTGTGAATT},$$

με κόστος στοίχισης $+1$, κόστος ασυμφωνίας -1 και κόστος στοίχισης με κενό -1 .

Σε όλους τους πίνακες που ακολουθούν, η ακολουθία v τοποθετείται στις γραμμές και η ακολουθία w στις στήλες. Με **χρώμα** σημειώνονται τα κελιά της διαδρομής οπισθοδρόμησης.

5.1 Ολική στοίχιση

5.1.1 Αναδρομική σχέση

Ο πίνακας S διαστάσεων $(n + 1) \times (m + 1)$ αρχικοποιείται με

$$S[i][0] = -i, \quad S[0][j] = -j,$$

δηλαδή κάθε μετακίνηση κατά μήκος της πρώτης γραμμής ή στήλης αντιστοιχεί στη στοίχιση ενός χαρακτήρα με κενό. Για κάθε επόμενο κελί ισχύει

$$S[i][j] = \max \begin{cases} S[i-1][j-1] + \sigma(v_i, w_j) & \text{(στοίχιση των δύο χαρακτήρων)} \\ S[i-1][j] - 1 & \text{(κενό στην } w) \\ S[i][j-1] - 1 & \text{(κενό στην } v) \end{cases}$$

όπου $\sigma(v_i, w_j) = +1$ αν $v_i = w_j$ και -1 διαφορετικά. Σε κάθε κελί αποθηκεύεται επιπλέον δείκτης προς το κελί από το οποίο προέκυψε η μέγιστη τιμή.

5.1.2 Παραδείγματα υπολογισμού

Κελί (A, G) , δηλαδή $i = 1, j = 1$. Οι χαρακτήρες δεν ταυτίζονται:

$$\max\{0 + (-1), (-1) - 1, (-1) - 1\} = \max\{-1, -2, -2\} = -1,$$

με διαγώνιο δείκτη.

Κελί (G, G) , δηλαδή $i = 5, j = 5$. Οι χαρακτήρες ταυτίζονται:

$$\max\{(-1) + 1, (-2) - 1, (-2) - 1\} = \max\{0, -3, -3\} = 0,$$

επίσης με διαγώνιο δείκτη. Πρόκειται για τη μέγιστη τιμή ολόκληρου του πίνακα.

5.1.3 Ο πίνακας

	—	G	T	G	T	G	A	A	T	T
—	0	-1	-2	-3	-4	-5	-6	-7	-8	-9
A	-1	-1	-2	-3	-4	-5	-4	-5	-6	-7
C	-2	-2	-2	-3	-4	-5	-5	-5	-6	-7
T	-3	-3	-1	-2	-2	-3	-4	-5	-4	-5
T	-4	-4	-2	-2	-1	-2	-3	-4	-4	-3
G	-5	-3	-3	-1	-2	0	-1	-2	-3	-4
G	-6	-4	-4	-2	-2	-1	-1	-2	-3	-4
G	-7	-5	-5	-3	-3	-1	-2	-2	-3	-4
T	-8	-6	-4	-4	-2	-2	-2	-3	-1	-2
G	-9	-7	-5	-3	-3	-1	-2	-3	-2	-2

Πίνακας 1: Πίνακας δυναμικού προγραμματισμού για την ολική στοίχιση των v και w .

5.1.4 Οπισθοδρόμηση και στοίχιση

Η οπισθοδρόμηση ξεκινά από το κάτω δεξιά κελί και τερματίζει στο $(0, 0)$:

$(9, 9) \rightarrow (8, 8) \rightarrow (7, 7) \rightarrow (6, 6) \rightarrow (5, 5) \rightarrow (4, 4) \rightarrow (3, 3) \rightarrow (3, 2) \rightarrow (2, 1) \rightarrow (1, 0) \rightarrow (0, 0)$.

Όλα τα βήματα είναι διαγώνια εκτός από δύο: το $(3, 3) \rightarrow (3, 2)$, που είναι οριζόντιο και αντιστοιχεί σε κενό στην v , και το $(1, 0) \rightarrow (0, 0)$, που είναι κάθετο και αντιστοιχεί σε κενό στην w . Προκύπτει η στοίχιση:

v :	A	C	T	-	T	G	G	G	T	G
w :	-	G	T	G	T	G	A	A	T	T
	-1	-1	+1	-1	+1	+1	-1	-1	+1	-1

Πίνακας 2: Βέλτιστη ολική στοίχιση και κόστος ανά στήλη.

Το συνολικό κόστος είναι 4 ταυτίσεις, 4 ασυμφωνίες και 2 κενά, δηλαδή $(+4) + (-4) + (-2) = -2$, τιμή που συμφωνεί με το κάτω δεξιά κελί του Πίνακα 1.

Τιμή βέλτιστης ολικής στοίχισης: -2 .

Ισοδύναμες λύσεις. Επειδή αρκετά κελιά του πίνακα δέχονται τη μέγιστη τιμή από περισσότερες από μία κατευθύνσεις, η βέλτιστη στοίχιση δεν είναι μοναδική. Υπάρχει ακόμη μία στοίχιση με το ίδιο κόστος -2 , που διαφέρει μόνο στις δύο πρώτες στήλες:

$$\begin{array}{l} v: \quad A \quad C \quad T \quad - \quad T \quad G \quad G \quad G \quad T \quad G \\ w: \quad G \quad - \quad T \quad G \quad T \quad G \quad A \quad A \quad T \quad T \end{array}$$

5.2 Τοπική στοίχιση

5.2.1 Διαφορές από την ολική στοίχιση

Η τοπική στοίχιση εντοπίζει το βέλτιστο ζεύγος υποακολουθιών και όχι τη στοίχιση των ακολουθιών σε όλο τους το μήκος. Οι διαφορές στον υπολογισμό συνοψίζονται στον Πίνακα 3.

	Ολική	Τοπική
Αρχικοποίηση	$-1, -2, -3, \dots$	όλα 0
Αναδρομή	$\max$ των τριών επιλογών	$\max$ των τριών και του 0
Θέση λύσης	κάτω δεξιά κελί	μέγιστο κελί οπουδήποτε
Τερματισμός οπισθοδρόμησης	στο $(0, 0)$	στο πρώτο μηδενικό κελί

Πίνακας 3: Σύγκριση των δύο αλγορίθμων δυναμικού προγραμματισμού.

Ο μηδενισμός των αρνητικών τιμών εκφράζει ότι, όταν το συσσωρευμένο κόστος γίνεται αρνητικό, είναι προτιμότερο να ξεκινήσει νέα τοπική στοίχιση παρά να μεταφερθεί η ποινή παρακάτω.

5.2.2 Ο πίνακας

	—	G	T	G	T	G	A	A	T	T
—	0	0	0	0	0	0	0	0	0	0
A	0	0	0	0	0	0	1	1	0	0
C	0	0	0	0	0	0	0	0	0	0
T	0	0	1	0	1	0	0	0	1	1
T	0	0	1	0	1	0	0	0	1	2
G	0	1	0	2	1	2	1	0	0	1
G	0	1	0	1	1	2	1	0	0	0
G	0	1	0	1	0	2	1	0	0	0
T	0	0	2	1	2	1	1	0	1	1
G	0	1	1	3	2	3	2	1	0	0

Πίνακας 4: Πίνακας δυναμικού προγραμματισμού για την τοπική στοίχιση των v και w .

5.2.3 Οπισθοδρόμηση και στοίχιση

Η μέγιστη τιμή του πίνακα είναι 3 και εμφανίζεται σε **δύο** κελιά της τελευταίας γραμμής, στις στήλες 3 και 5. Ξεκινώντας από το κελί της στήλης 3, η οπισθοδρόμηση ακολουθεί τρία διαγώνια βήματα μέχρι το πρώτο μηδενικό κελί:

$$(9, 3) = 3 \rightarrow (8, 2) = 2 \rightarrow (7, 1) = 1 \rightarrow (6, 0) = 0 \text{ (τερματισμός)}.$$

$v:$	G	T	G
$w:$	G	T	G
	+1	+1	+1

Πίνακας 5: Βέλτιστη τοπική στοίχιση: θέσεις 7–9 της v με θέσεις 1–3 της w .

Η στοίχιση αποτελείται από τρεις ταυτίσεις, χωρίς ασυμφωνίες ή κενά.

Τιμή βέλτιστης τοπικής στοίχισης: 3, με στοίχιση $GTG \leftrightarrow GTG$.

Ισοδύναμη λύση. Το δεύτερο κελί με τιμή 3 οδηγεί στην ίδια υποακολουθία GTG της v , στοιχισμένη όμως με τις θέσεις 3–5 της w . Αυτό οφείλεται στο ότι η w ξεκινά με GTGTG και επομένως

περιέχει το μοτίβο GTG δύο φορές, σε επικαλυπτόμενες θέσεις. Ο αλγόριθμος δεν διαθέτει κριτήριο για να προτιμήσει τη μία εμφάνιση έναντι της άλλης.

5.3 Συμπεράσματα

Παρατηρούμε ότι η βέλτιστη ολική στοίχιση έχει αρνητικό κόστος (-2), ενώ η βέλτιστη τοπική έχει θετικό ($+3$). Η φαινομενική αντίφαση εξηγείται από τη δομή των δύο ακολουθιών: υπάρχει ένα σύντομο κοινό μοτίβο, το GTG, ενώ το υπόλοιπο μήκος τους παρουσιάζει μικρή ομοιότητα. Η ολική στοίχιση υποχρεούται να στοιχίσει και τις ανόμοιες περιοχές, επιβαρυνόμενη με τις αντίστοιχες ποινές, ενώ η τοπική απομονώνει μόνο το τμήμα που συνεισφέρει θετικά.

Η παρατήρηση αυτή αιτιολογεί τη χρήση τοπικής στοίχισης στα εργαλεία αναζήτησης ομοιότητας, όπως το BLAST στο οποίο κατά την αναζήτηση ενός μοτίβου σε μεγάλη βάση δεδομένων, το ζητούμενο είναι ο εντοπισμός συντηρημένων περιοχών και όχι η στοίχιση των ακολουθιών σε όλο τους το μήκος.

6 Ερώτημα 6

6.1 Αλγόριθμος με suffix tree

6.1.1 Διατύπωση του προβλήματος

Δίνεται συμβολοσειρά κειμένου T μήκους n . Ζητείται δομή δεδομένων που κατασκευάζεται σε χρόνο $O(n)$ και επιτρέπει, για κάθε συμβολοσειρά P μήκους m και κάθε τιμή k με $1 \leq k \leq n$, τον έλεγχο σε χρόνο $O(m)$ του κατά πόσον το P εμφανίζεται στο T σε θέση αυστηρά μικρότερη του k .

Ο περιορισμός $O(m)$ αφορά τον χρόνο απόκρισης ο οποίος πρέπει να εξαρτάται αποκλειστικά από το μήκος του μοτίβου και όχι από το μέγεθος του κειμένου ούτε από το πλήθος των εμφανίσεων του P σε αυτό.

6.1.2 Βασική ιδιότητα του suffix tree

Το "suffix tree" του T περιέχει, ως μονοπάτια από τη ρίζα, όλα τα επιθέματα του T . Κάθε φύλλο αντιστοιχεί σε ένα επίθεμα και αποθηκεύει τη θέση από την οποία αυτό ξεκινά.

Η ιδιότητα στην οποία στηρίζεται ο αλγόριθμος είναι η εξής: κάθε υποσυμβολοσειρά του T αποτελεί πρόθεμα κάποιου επιθέματος του T . Συνεπώς, η αναζήτηση του P ανάγεται σε κάθοδο από τη ρίζα ακολουθώντας διαδοχικά τους χαρακτήρες του, με κόστος ένα βήμα ανά χαρακτήρα.

Έστω v ο κόμβος στον οποίο καταλήγει η κάθοδος αυτή. Τα φύλλα του υποδέντρου με ρίζα το v αντιστοιχούν ακριβώς στα επιθέματα που έχουν το P ως πρόθεμα, δηλαδή ακριβώς στις θέσεις εμφάνισης του P μέσα στο T .

6.1.3 Το εμπόδιο και η άρση του

Η προφανής προσέγγιση κάθοδος με το P και στη συνέχεια συλλογή όλων των θέσεων από τα φύλλα του υποδέντρου δεν ικανοποιεί τον περιορισμό. Αν το P εμφανίζεται occ φορές στο T , το υποδέντρο έχει occ φύλλα και ο χρόνος γίνεται $O(m + occ)$, ποσότητα που εξαρτάται από το κείμενο.

Παρατηρούμε ωστόσο ότι δεν απαιτούνται όλες οι θέσεις εμφάνισης, παρά μόνο η *μικρότερη* από αυτές: αν ούτε η πρωιμότερη εμφάνιση δεν προηγείται της θέσης k , τότε καμία δεν προηγείται. Η ποσότητα αυτή μπορεί να προϋπολογιστεί κατά την προεπεξεργασία. Ορίζουμε, για κάθε κόμβο v του δέντρου,

$$\min(v) = \min\{i : \text{το φύλλο με θέση } i \text{ ανήκει στο υποδέντρο με ρίζα το } v\}.$$

6.1.4 Ο αλγόριθμος

Αλγόριθμος 1: Προεπεξεργασία του κειμένου T

Είσοδος: Συμβολοσειρά T μήκους n

Έξοδος: suffix tree με τιμές $\min(v)$ σε κάθε κόμβο

- 1 Κατασκεύασε το δέντρο επιθεμάτων $\mathcal{T}$ του T με τον αλγόριθμο του Ukkonen;
 - 2 **foreach** κόμβο v του $\mathcal{T}$ σε διάσχιση από τα φύλλα προς τη ρίζα **do**
 - 3 **if** το v είναι φύλλο **then**
 - 4 $\min(v) \leftarrow$ η θέση του επιθέματος που αντιστοιχεί στο v ;
 - 5 **else**
 - 6 $\min(v) \leftarrow \min\{\min(u) : u \text{ παιδί του } v\}$;
 - 7 **return** $\mathcal{T}$;
-

Αλγόριθμος 2: Έλεγχος εμφάνισης του P πριν από τη θέση k

Είσοδος: Μοτίβο P μήκους m , θέση k , προεπεξεργασμένο δέντρο $\mathcal{T}$

Έξοδος: TRUE αν το P εμφανίζεται στο T σε θέση $i < k$, αλλιώς FALSE

- 1 $v \leftarrow$ ρίζα του $\mathcal{T}$;
 - 2 Κατέβα από το v ακολουθώντας διαδοχικά τους χαρακτήρες $P[1], P[2], \dots, P[m]$;
 - 3 **if** η κάθοδος διακοπεί πριν καταναλωθεί ολόκληρο το P **then**
 - 4 **return** FALSE ; // το P δεν εμφανίζεται καθόλου στο T
 - 5 **if** η κάθοδος τερματίσει στο εσωτερικό ακμής **then**
 - 6 $v \leftarrow$ ο κόμβος στο κάτω άκρο της ακμής αυτής;
 - 7 **return** $\min(v) < k$;
-

6.1.5 Ορθότητα

Η ορθότητα προκύπτει από δύο διαπιστώσεις.

Το υποδέντρο του v περιέχει ακριβώς τις εμφανίσεις του P . Η κάθοδος με τους χαρακτήρες του P οδηγεί στο σημείο του δέντρου του οποίου το μονοπάτι από τη ρίζα ισούται με P . Τα φύλλα κάτω από το σημείο αυτό αντιστοιχούν στα επιθέματα που έχουν το P ως πρόθεμα. Ένα επίθεμα που ξεκινά στη θέση i έχει το P ως πρόθεμα ακριβώς όταν το P εμφανίζεται στο T στη θέση i . Επομένως το σύνολο των θέσεων που αποθηκεύονται στα φύλλα του υποδέντρου ταυτίζεται με το σύνολο των εμφανίσεων του P .

Αν η κάθοδος τερματίσει στο εσωτερικό ακμής, το σύνολο των φύλλων κάτω από το σημείο τερματισμού ταυτίζεται με το σύνολο των φύλλων του κόμβου στο κάτω άκρο της ακμής, καθώς οι ακμές του δέντρου επιθεμάτων είναι συμπιεσμένες και δεν εμπεριέχουν διακλαδώσεις. Η αντικατάσταση του βήματος 4 είναι συνεπώς ασφαλής.

Ο έλεγχος $\min(v) < k$ είναι ισοδύναμος με το ζητούμενο. Έστω I το σύνολο των εμφανίσεων του P . Ζητείται να διαπιστωθεί αν υπάρχει $i \in I$ με $i < k$. Επειδή $\min(v) = \min I$, ισχύει

$$(\exists i \in I : i < k) \iff \min I < k \iff \min(v) < k.$$

Η ευθεία κατεύθυνση προκύπτει διότι $\min I \leq i < k$, και η αντίστροφη διότι το ίδιο το $\min I$ αποτελεί στοιχείο του I . Στην περίπτωση που το P δεν εμφανίζεται καθόλου, ο έλεγχος δεν εκτελείται· ο αλγόριθμος επιστρέφει FALSE στο βήμα 3.

6.1.6 Ανάλυση πολυπλοκότητας

Προεπεξεργασία. Ο αλγόριθμος του Ukkonen κατασκευάζει το δέντρο επιθεμάτων σε χρόνο $O(n)$ για αλφάβητο σταθερού μεγέθους. Ο υπολογισμός των τιμών $\min(v)$ απαιτεί μία διάσχιση του δέντρου· κάθε κόμβος επισκέπτεται μία φορά και το κόστος του είναι ανάλογο του πλήθους των παιδιών του. Το άθροισμα των βαθμών όλων των κόμβων ισούται με το πλήθος των ακμών, το οποίο είναι $O(n)$, αφού το δέντρο επιθεμάτων του T έχει το πολύ n φύλλα και συνεπώς το πολύ $n - 1$ εσωτερικούς κόμβους. Η συνολική προεπεξεργασία είναι επομένως $O(n)$.

Απάντηση σε ερώτημα. Η κάθοδος καταναλώνει έναν χαρακτήρα του P ανά βήμα, δηλαδή το πολύ m βήματα. Ο εντοπισμός του κόμβου στο κάτω άκρο της ακμής, η ανάγνωση της τιμής $\min(v)$ και η σύγκριση με το k εκτελούνται σε σταθερό χρόνο. Συνολικά $O(m)$, ανεξάρτητα από το n και από το πλήθος των εμφανίσεων του P .

Χώρος. Το δέντρο επιθεμάτων καταλαμβάνει $O(n)$ χώρο. Η αποθήκευση μίας ακέραιης τιμής ανά κόμβο προσθέτει $O(n)$, οπότε η συνολική απαίτηση παραμένει $O(n)$.

Παρατήρηση επί του αλφαβήτου. Ο ισχυρισμός ότι η επιλογή του κατάλληλου παιδιού σε κάθε κόμβο γίνεται σε σταθερό χρόνο προϋποθέτει αλφάβητο σταθερού μεγέθους — υπόθεση που ικανοποιείται στις βιολογικές εφαρμογές, όπου το αλφάβητο αποτελείται από τέσσερα νουκλεοτίδια ή είκοσι αμινοξέα — ή, εναλλακτικά, αποθήκευση των παιδιών κάθε κόμβου σε πίνακα κατακερματισμού. Χωρίς κάποια από τις δύο αυτές υποθέσεις, ο χρόνος απόκρισης επιβαρύνεται με λογαριθμικό παράγοντα ως προς το μέγεθος του αλφαβήτου.

6.2 Δεικτοδότηση βιολογικών δεδομένων: Bloom filters, suffix arrays και BWT

Αναφορές

- [1] J. Abramson *et al.*, “Accurate structure prediction of biomolecular interactions with AlphaFold 3,” *Nature*, τ. 630, σσ. 493–500, 2024. <https://doi.org/10.1038/s41586-024-07487-w>
- [2] Z. Lin *et al.*, “Evolutionary-scale prediction of atomic-level protein structure with a language model,” *Science*, τ. 379, τχ. 6637, σσ. 1123–1130, 2023. <https://doi.org/10.1126/science.ade2574>
- [3] M. Mirdita *et al.*, “ColabFold: making protein folding accessible to all,” *Nature Methods*, τ. 19, σσ. 679–682, 2022.
- [4] O. Sladký, P. Veselý, K. Břinda, “From Superstring to Indexing: a space-efficient index for unconstrained k-mer sets using the Masked Burrows–Wheeler Transform (MBWT),” *Bioinformatics Advances*, τ. 6, τχ. 1, vbaf290, 2025. <https://doi.org/10.1093/bioadv/vbaf290>